

Đề thi cuối Module 1 — Dự án: Trợ lý hỏi-đáp theo luật

Final Project: A Rule-Based Q&A Assistant

Hình thức: Bài tập lớn làm ở nhà (take-home). **Thời lượng đề nghị:** 5–7 ngày.

Điểm: 100. **Nộp:** một tệp `solution.py` (+ tệp `knowledge_base.json` của bạn + `chat_log.txt` minh hoạ).

1. Bối cảnh | Scenario

Khoa CNTT muốn có một **trợ lý ảo** trả lời các câu hỏi thường gặp của sinh viên (giờ mở cửa phòng máy, mật khẩu WiFi, cách nộp bài...). Chưa cần tới Machine Learning — ở Module 1 ta xây một **“AI theo luật” (rule-based)**: trợ lý so khớp câu hỏi của người dùng với một **kho tri thức** do con người soạn sẵn, rồi chọn câu trả lời phù hợp nhất.

Đây chính là cách các hệ chuyên gia (expert systems) và chatbot đời đầu hoạt động — nền tảng lịch sử của Trí tuệ nhân tạo mà bạn sẽ học sâu ở các chương sau.

Bạn sẽ tự tay xây dựng trợ lý đó **chỉ bằng kiến thức Module 1**: chuỗi, điều khiển, hàm, lập trình hướng đối tượng và xử lý tệp.

2. Trợ lý hoạt động thế nào? | How it works

- Người dùng gõ một câu, ví dụ: “*cho mình hỏi mật khẩu wifi với*”.
- Trợ lý **chuẩn hoá** câu (bỏ dấu, chữ thường, bỏ dấu câu) → cho mình hỏi mật khẩu wifi voi.
- Tách từ** thành tập: {cho, mình, hỏi, mật, khẩu, wifi, voi}.
- So với từng **intent** (ý định) trong kho tri thức bằng **điểm khớp từ khoá**:

$$\text{keyword_score} = (\text{số từ của MẪU xuất hiện trong câu}) / (\text{tổng số từ của MẪU})$$

Ví dụ mẫu “mat khẩu wifi” có 3 từ, câu chứa cả 3 → điểm $3/3 = 1.0$.

- Chọn intent điểm cao nhất. Nếu điểm $\geq$ threshold (mặc định **0.5**) → trả lời; ngược lại trả câu **fallback** (“mình chưa hiểu...”).
- Ngoài tra cứu FAQ, trợ lý còn có **kỹ năng tính toán**: gõ $12 + 30 \rightarrow 12 + 30 = 42$.

Trợ lý điều phối nhiều **Skill** (kỹ năng) theo thứ tự — đây là chỗ bạn dùng **kế thừa & đa hình**: mỗi Skill là một lớp con, tự cài đặt cách xử lý riêng.

3. Kiến thức được kiểm tra | Module coverage

Phần	Nội dung	Bài liên quan
A	Xử lý chuỗi: bỏ dấu, chuẩn hoá, tách từ, tính điểm khớp (set)	Bài 2, 3, 4
B	Lớp Intent, KnowledgeBase; nạp JSON; bắt ngoại lệ tệp	Bài 5, 6
C	Lớp trừu tượng Skill + FaqSkill, MathSkill (kế thừa, đa hình)	Bài 5
D	Lớp Assistant: hội thoại, thống kê, ghi nhật ký ra tệp	Bài 5, 6
E	Giao diện dòng lệnh + kho tri thức của riêng bạn	Bài 1, 3, 6

4. Yêu cầu chi tiết | Specification

Hoàn thiện tệp `starter.py` (đổi tên/lưu thành `solution.py`). **Không đổi tên hàm, lớp, tham số đã cho** — bộ kiểm thử dựa vào đúng các tên này.

PHẦN A — Tiện ích văn bản (15đ)

Hàm	Mô tả	Ví dụ
<code>strip_accents(text)</code>	Bỏ dấu tiếng Việt, đ/Đ → d/D	"Chào Đại" → "Chao Dai"
<code>normalize(text)</code>	Bỏ dấu → chữ thường → bỏ dấu câu → gộp khoảng trắng	"Wi-Fi của Khoa?" → "wi fi cua khoa"
<code>tokenize(text)</code>	Tách câu đã chuẩn hoá thành danh sách từ	"Mấy giờ?" → ["may", "gio"]
<code>keyword_score(text_tokens, pattern_tokens)</code>	Đã hệ từ của mẫu có trong câu (0–1); mẫu rỗng → 0.0	xem mục 2

Gợi ý `strip_accents`: dùng `unicodedata.normalize("NFD", text)` rồi loại các ký tự có `category == "Mn"`. Riêng đ phải thay bằng `str.replace` trước.

PHẦN B — Intent & KnowledgeBase (25đ)

Intent — đóng gói dữ liệu bằng thuộc tính nội bộ `_tag`, `_patterns`, `_responses`, truy cập qua **property** `tag`, `patterns`, `responses`:

- `score(user_tokens)` → điểm khớp **cao nhất** giữa câu (tập từ) và các mẫu.

- `reply(index=0)` → câu trả lời, **xoay vòng** theo index (`index % số_câu`).

KnowledgeBase — chứa danh sách Intent:

- `__len__` → số intent.
- `load_json(path)` (*classmethod*) → tạo KB từ JSON; **ném** `KnowledgeBaseError` khi: tệp không tồn tại / JSON hỏng / thiếu khoá "intents".
- `find_best(user_text)` → (intent, score) khớp nhất; (None, 0.0) nếu không khớp gì.

PHẦN C — Skill, kế thừa & đa hình (25đ)

- Skill là **lớp trừu tượng** (đã cho) với phương thức trừu tượng `handle(text)`.
- `FaqSkill(Skill)`: `handle` dùng `KnowledgeBase.find_best`; chỉ trả lời khi `score ≥ threshold`, ngược lại trả None (để nhường kỹ năng khác).
- `MathSkill(Skill)`: `handle` nhận chuỗi "`a op b`" (cách nhau bởi dấu cách, $op \in \{+, -, *, /\}$); trả kết quả, ví dụ "`10 / 4`" → "`10 / 4 = 2.5`"; chia 0 → "Không thể chia cho 0!"; không phải phép tính → None.
- Hàm phụ `is_number(token)` → True/False (dùng try/except).

Quy ước hiển thị: nếu kết quả là số nguyên (vd 42.0) thì in 42, ngược lại giữ thập phân.

PHẦN D — Assistant & nhật ký (20đ)

`Assistant(skills, fallback=FALLBACK)` — lưu `_skills`, `_fallback`, `_history` (list các cặp (role, text) với `role ∈ {"user", "bot"}`):

- `respond(text)` → hỏi **lần lượt** từng skill, skill nào trả khác None thì dùng; hết mà không ai trả → dùng fallback. Nhớ **ghi lịch sử**: ("user", text) rồi ("bot", reply).
- `stats()` → {"messages": <số câu hỏi>, "answered": <số câu trả được>, "fallback": <số lần không hiểu>}.
- `reset()` → xoá lịch sử.
- `save_log(path)` → ghi tệp UTF-8, mỗi dòng Bạn: ... hoặc Trợ lý:

PHẦN E — CLI + kho tri thức (15đ)

- Phần CLI (`build_assistant`, `chat_loop`, `main...`) **đã viết sẵn**. Bạn chỉ cần để code phần A–D chạy đúng là `python3 solution.py` hoạt động.
- **Tự bổ sung kho tri thức**: thêm ít nhất **3 intent mới** của riêng bạn vào `knowledge_base.json` (mỗi intent ≥ 3 mẫu câu và ≥ 1 câu trả lời) — ví dụ: học phí, lịch thi, mượn sách thư viện... (5đ trong 15đ phần E).

5. Cách chạy & tự kiểm tra | Run & self-check

1) Chạy bộ kiểm thử (mục tiêu: "ALL TESTS PASSED")

```
python3 test_solution.py
```

2) Trò chuyện thử với trợ lý

```
python3 solution.py
```

Bạn: mật khẩu wifi

Trợ lý: Mật khẩu WiFi của Khoa là: ...

Bạn: 15 + 27

Trợ lý: 15 + 27 = 42

Bạn: /save

Bạn: /quit

Bộ test test_solution.py là **một phần** của thang điểm, **không phải toàn bộ**. Code chạy qua test nhưng câu trả lời, không docstring, đặt tên tùy tiện vẫn bị trừ điểm chất lượng.

6. Thang điểm | Rubric (100đ)

Tiêu chí	Điểm
Tính đúng đắn (qua test + chấm tay)	75
• Phần A — xử lý chuỗi	15
• Phần B — Intent & KnowledgeBase	25
• Phần C — Skill, kế thừa, đa hình	25
• Phần D — Assistant & file I/O	10
Phần E — CLI chạy được + ≥ 3 intent tự thêm	10
Chất lượng code (đặt tên, docstring, gọn gàng, không lặp)	10
Báo cáo ngắn BAOCÁO.md (mục 7)	5
Điểm thưởng (tối đa +10, không vượt 100)	+10

Gợi ý điểm thưởng (chọn 1–2):

- Thêm một **Skill mới** thực sự (vd TimeSkill trả giờ hiện tại bằng datetime; HelpSkill liệt kê việc trợ lý làm được).
- **Cải thiện khớp từ khoá** để giảm khớp nhầm (xem mục 7): loại bỏ *stopword* (các từ quá phổ biến như “cho, mình, với”), hoặc dùng độ đo Jaccard.
- Trả lời **đa dạng** (xoay vòng/ngẫu nhiên các câu trong responses).

- Lưu nhật ký ra **CSV** kèm điểm khớp của mỗi lượt.
-

7. Báo cáo ngắn & câu hỏi tư duy | Reflection (5đ + chống sao chép)

Nộp kèm tệp `BAOCAO.md` (khoảng 1 trang) trả lời **bằng lời của bạn**:

1. Vì sao `find_best` trả (`None`, `0.0`) lại tốt hơn là cứ trả intent điểm cao nhất dù rất thấp?
2. **Thử nghiệm**: gõ cho trợ lý câu “*bạn ăn cơm chưa*”. Nó trả lời gì? **Giải thích vì sao** lại khớp nhầm (gợi ý: nhìn công thức `keyword_score` và các mẫu của intent “greeting”). Bạn sẽ sửa thế nào?
3. Vì sao đặt `MathSkill` **trước** `FaqSkill` trong danh sách? Đổi thứ tự thì sao?
4. `strip_accents` giúp gì cho việc khớp câu hỏi của người dùng?

Trả lời mục 7 cho thấy bạn **hiểu** code mình viết. GV có thể hỏi vấn đáp 3–5 phút dựa trên chính báo cáo này.

8. Quy định nộp bài | Submission

1. Tạo thư mục `Exam_TroLy_<MSSV>` chứa:
 - `solution.py` — bài làm.
 - `knowledge_base.json` — kho tri thức (đã thêm intent của bạn).
 - `chat_log.txt` — một phiên trò chuyện mẫu (dùng `/save`).
 - `BAOCAO.md` — báo cáo ngắn.
 2. Nén thư mục thành `.zip` và nộp lên LMS trước hạn.
 3. **Liên chính học thuật**: được tra cứu tài liệu, nhưng phải tự viết và **hiểu** bài. Bài giống nhau bất thường hoặc không trả lời được câu hỏi vấn đáp sẽ bị xử lý theo quy chế.
-

9. Lời khuyên | Tips

- Làm theo thứ tự **A → B → C → D**, chạy test sau mỗi phần (test in rõ phần nào đậu).
 - Viết hàm nhỏ, thử ngay trong Python shell: `from solution import normalize; normalize("Chào!")`.
 - Xử lý trường hợp biên: chuỗi rỗng, mẫu rỗng, danh sách responses 1 phần tử.
 - Luôn mở tệp bằng `with open(... encoding="utf-8")` và bắt lỗi bằng `try/except`.
 - Đọc kỹ thông báo lỗi đỏ — nó chỉ đúng dòng sai. Báo lỗi là bạn, không phải kẻ thù.
-

Chúc các bạn làm bài tốt và vui với “AI đời đầu” của chính mình!